

Network Security: Protocols

Assumed Knowledge and Recommended Reading

We assume you have basic knowledge of networks - Equivalent to the chapters below from **Computer Networking: A Top-down Approach – 6th Edition** by James Kurose and Keith Ross.

- **Chapter 1** – Computer Network And The Internet
- **Chapter 2** – Application Layer
- **Chapter 3** – Transport Layer
- **Chapter 4** – The Network Layer
- **Chapter 5** – The Link Layer

Recommended readings are below.

Computer Networking: A Top-down Approach – 6th Edition by James Kurose and Keith Ross

- **Chapter 8** - Security In Computer Networks

Cryptography and Network Security (7th Edition - William Stallings)

- **Chapter 17** - Transport-Level Security
- **Chapter 18** - IP Security

These lecture notes are given to you to assist with understanding the lecture content better. This content is prepared based on the above book chapters. You are not allowed to upload this material to any internet source or share it with anyone else.

1 Networking Overview

Our technological infrastructure, applications, and services are deeply integrated into networked environments. In fact, the reliance on networks has grown significantly, especially after the COVID-19 pandemic, which accelerated the shift to remote work and digital services. Networks now facilitate critical operations, from managing infrastructure like power grids to conducting business remotely. Both businesses and governments increasingly depend on secure networking environments, often establishing private networks and employing VPNs to maintain secure connections from remote locations.

Network security plays a critical role in protecting these systems, as most attacks target networked environments. These can include remote exploits, such as SQL injection attacks on web applications. Network security aims to defend systems against such threats while operating in a highly interconnected landscape. Notably, attackers often possess distinct advantages over defenders, with any device—such as a browser, CCTV camera, or IoT sensor—potentially becoming part of a botnet. This asymmetry puts defenders at a structural disadvantage. Given the complexity and breadth of networking, it is a field that demands focused study. In this unit, we introduce the foundational concepts and building blocks of network security.

1.1 The Internet

The Internet interconnects hundreds of millions of computing devices globally, known as hosts or end systems, with nearly 850 million connected as of July 2011, excluding intermittently connected devices like smartphones and laptops, bringing the estimated total of Internet users to around 2 billion. These devices are linked through a network of various communication links, from coaxial cables to optical fibers, each with different transmission rates known as bandwidth. Packet switches, primarily routers and link-layer switches, route data across these links. Access to the Internet is provided through Internet Service Providers (ISPs), ranging from residential and corporate to university ISPs, along with public WiFi services in locations like airports and cafes.

To understand how data is transmitted and routed across these vast networks, it is necessary to understand the TCP/IP protocol stack, which forms the foundation of modern Internet communication. Though the OSI (Open Systems Interconnection) model was historically significant in standardizing network architecture, it is largely theoretical today. The TCP/IP model, with its four layers (application, transport, network, and network interface), is the practical framework that governs today's Internet traffic, offering the blueprint for how data is packaged, transmitted, and received across interconnected systems worldwide.

1.2 The TCP/IP Model

The four layers of the TCP/IP model:

1. Application layer
2. Transport layer
3. Network layer
4. Network interface layer (it can be separated as data link layer and physical layer)

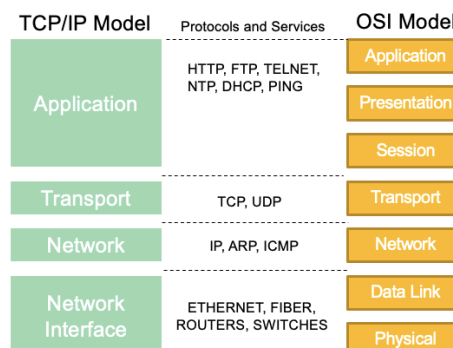


Figure 1: The TCP/IP Model

What is a Protocol? Protocols are everywhere. It controls sending, receiving of messages. Such as HTTP (Web), streaming video, Skype, TCP, IP, WiFi, 4G, Ethernet. It is probably easiest to understand the notion of a computer network protocol by first considering some human analogies, since we humans execute protocols all of the time. Consider what you do when you want to ask someone for the time of day. A network protocol is similar to a human protocol, except that the entities exchanging messages and taking actions are hardware or software components of some device. All activity in the Internet that involves two or more communicating remote entities is governed by a protocol. For example, protocols in routers determine a packet's path from source to destination.

A typical exchange is shown in Figure 2. Human protocol dictates that one first offer a greeting (the first “Hi”) to initiate communication with someone else. The typical response to a “Hi” is a returned “Hi” message. Implicitly, one then takes a cordial “Hi” response as an indication that one can proceed and ask for the time of day. For the network protocol, PC sends a TCP connection request to the server, and the server sends back a TCP connection response. The PC or user then wants to access this URL, the server will response with the requested files. This explains key points about distributed entities exchanging messages governed by protocols, with time progressing downward, and reviews the definition of a protocol, which includes the format, the order in which messages are sent and received, and the actions taken.

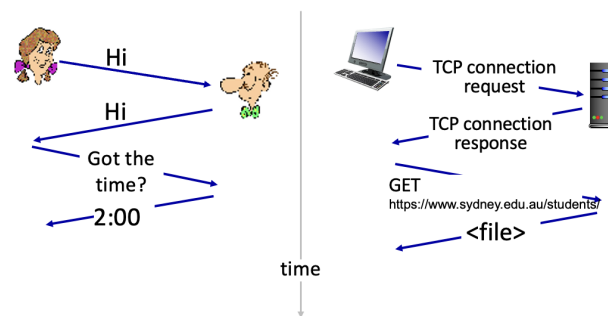


Figure 2: A human protocol and a computer network protocol

Given the human and networking examples above, the exchange of messages and the actions taken when these messages are sent and received are the key defining elements of a protocol: A protocol defines the format and the order of messages exchanged between two or more communicating entities, as well as the actions taken on the transmission and/or receipt of a message or other event.

1.2.1 Application Layer

The application layer is where network applications and their application-layer protocols reside. The Internet's application layer includes many protocols, such as the HTTP protocol (which provides for Web document request and transfer), SMTP (which provides for the transfer of e-mail messages), and FTP (which provides for the transfer of files between two end systems). We'll see that certain network functions, such as the translation of human-friendly names for Internet end systems like www.ietf.org to a 32-bit network address, are also done with the help of a specific application-layer protocol, namely, the domain name system (DNS).

An application-layer protocol is distributed over multiple end systems, with the application in one end system using the protocol to exchange packets of information with the application in another end system. We'll refer to this packet of information at the application layer as a message.

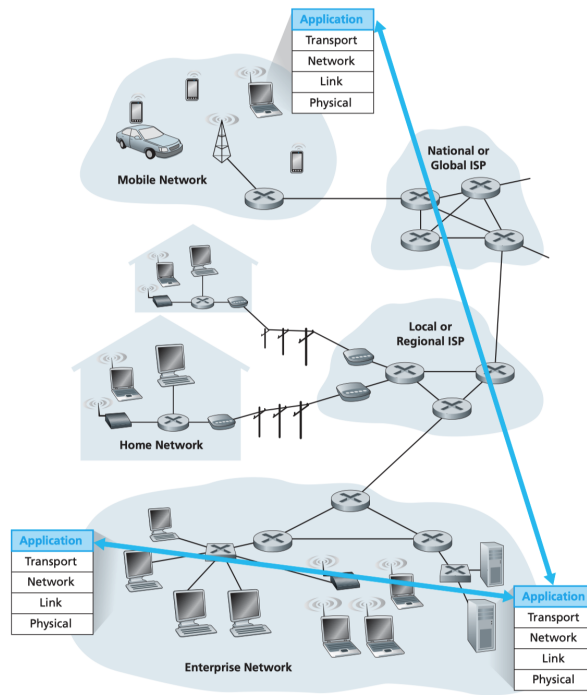


Figure 3: Communication for a network application takes place between end systems at the application layer

Sockets A process sends messages into, and receives messages from, the network through a software interface called a socket. Let's consider an analogy to help us understand processes and sockets. A process is analogous to a house and its socket is analogous to its door. When a process wants to send a message to another process on another host, it shoves the message out its door (socket). This sending process assumes that there is a transportation infrastructure on the other side of its door that will transport the message to the door of the destination process. Once the message arrives at the destination host, the message passes through the receiving process's door (socket), and the receiving process then acts on the message.

Figure 4 illustrates socket communication between two processes that communicate over the Internet. (Figure 4 assumes that the underlying transport protocol used by the processes is the Internet's TCP protocol.) As shown in this figure, a socket is the interface between the application layer and the transport layer within a host. It is also referred to as the Application Programming Interface (API) between the application and the network, since the socket is the programming interface with which network applications are built.

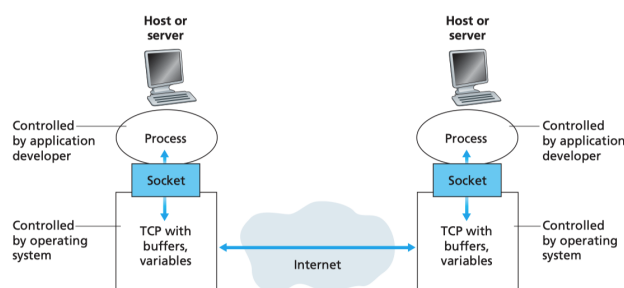


Figure 4: Application processes, sockets, and underlying transport protocol

1.2.2 Transport Layer

The Internet's transport layer transports application-layer messages between application endpoints. It provides logical communication between application processes running on different hosts. In the Internet there are two transport protocols, TCP and UDP, either of which can transport application-layer messages. TCP provides a connection-oriented service to its applications. This service includes guaranteed delivery of application-layer messages to the destination and flow control (that is, sender/receiver speed matching). TCP also breaks long messages into shorter segments and provides a congestion-control mechanism, so that a source throttles its transmission rate when the network is congested. The UDP protocol provides a connectionless service to its applications. This is a no-frills service that provides no reliability, no flow control, and no congestion control. We refer to a transport-layer packet as a *segment*. In general, the sender breaks application messages into segments and then passes them to the network layer. The receiver reassembles segments into messages and passes them to the application layer.

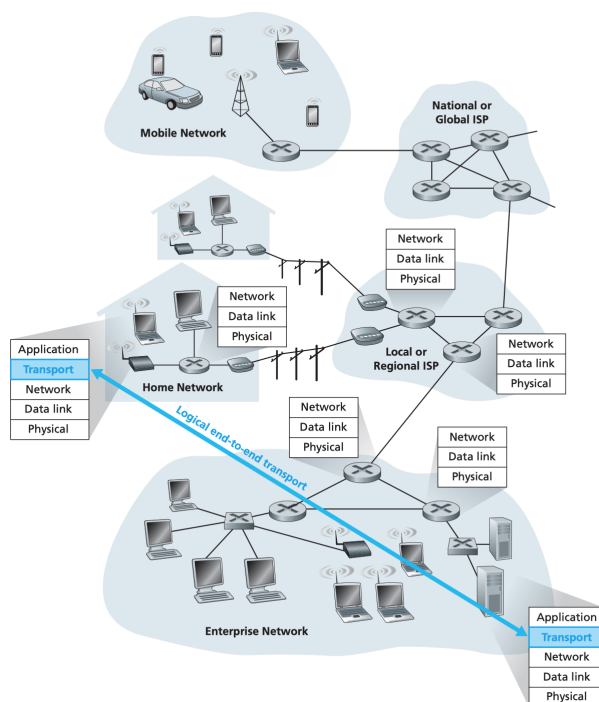


Figure 5: The transport layer provides logical rather than physical communication between application processes

UDP: User Datagram Protocol UDP is primarily utilized in applications like streaming multi-media, which are loss-tolerant and rate-sensitive, as well as in protocols such as the Domain Name System, Simple Network Management Protocol, and HTTP/3. It is considered a “no frills” protocol due to its potential for lost segments and out-of-order delivery, operating on a best-effort service model—essentially, “send and hope for the best.” Unlike TCP, UDP does not involve handshaking between sending and receiving transport-layer entities before sending a segment, making it connectionless. We will compare this later with TCP, which employs a three-way handshake, enhancing its reliability. Despite these characteristics, UDP can function effectively even when network service is compromised, aiding in reliability through mechanisms like the UDP checksum for error detection. For applications requiring faster hypertext transfer protocols with lower latency, such as HTTP/3, additional functionality can be built on top of UDP at the application layer to ensure both security and efficiency. HTTP/3 is notable for being the first HTTP protocol that does not rely on TCP.

TCP: Transmission Control Protocol TCP is considered connection-oriented because it requires a preliminary “handshake” between two application processes before data transmission can begin, establishing the parameters for the data transfer. This process is integral to setting up a TCP connection, which is always point-to-point, involving just a single sender and a single receiver. TCP connections support full-duplex service, allowing data to flow simultaneously between two processes, such as from Process A to Process B and vice versa. Additionally, TCP employs pipelining, which permits multiple transmitted but unacknowledged segments to be outstanding at any time, facilitating congestion and flow control through window sizing.

The **three-way handshake** (Figure 6) typical of TCP involves sending three segments between the hosts: the first two segments initiate the connection and carry no payload, while the third segment may contain data. This procedure confirms the connection before any substantial data exchange occurs. Once these three steps have been completed, the client and server hosts can send segments containing data to each other. TCP also manages the maximum segment size (MSS), which limits the amount of data in a segment, and provides flow-control services to prevent the sender from overwhelming the receiver’s buffer. Acknowledgments in TCP are cumulative, covering all data up to the first missing byte in the stream, enhancing the reliability of the data transfer.

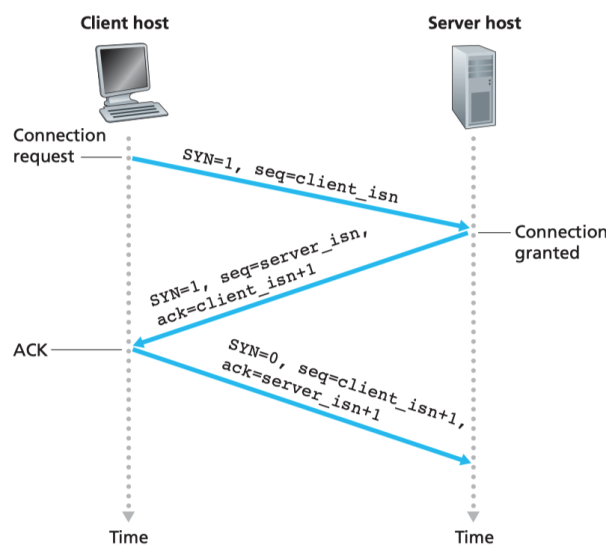


Figure 6: TCP three-way handshake: segment exchange

TCP three-way handshake:

1. A client node sends a SYN data packet over an IP network to a server on the same or an external network. The objective of this packet is to ask/infer if the server is open for new connections.
2. The target server must have open ports that can accept and initiate new connections. When the server receives the SYN packet from the client node, it responds and returns a confirmation receipt – the ACK packet or SYN/ACK packet.
3. The client node receives the SYN/ACK from the server and responds with an ACK packet.

1.2.3 Network Layer

The Internet’s network layer is responsible for moving network-layer packets known as datagrams from one host to another. The Internet transport-layer protocol (TCP or UDP) in a source host

passes a transport-layer segment and a destination address to the network layer, just as you would give the postal service a letter with a destination address. The network layer then provides the service of delivering the segment to the transport layer in the destination host.

The Internet's network layer includes the celebrated IP Protocol, which defines the fields in the datagram as well as how the end systems and routers act on these fields. There is only one IP protocol, and all Internet components that have a network layer must run the IP protocol.

The Internet's network layer also contains routing protocols such as RIP, OSPF, and BGP that determine IP datagrams' routes between sources and destinations. Overall, the Internet is a network of networks, and within a network, the network administrator can run any routing protocol desired. The routing protocol implemented on routers examines header fields in all IP datagrams passing through it and moves datagrams from input ports to output ports to facilitate their transfer along the end-to-end path.

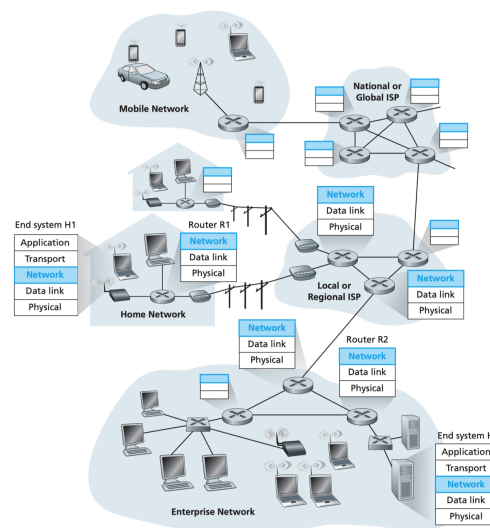


Figure 7: The Network Layer

As shown in Figure 8, the Internet's network layer has three major components. The first component is the IP protocol, the topic of this section. The second major component is the routing component, which determines the path a datagram follows from source to destination. For example, we have path-selection algorithms that are implemented in routing protocols (such as Open Shortest Path First and Border Gateway Protocol) or software-defined networking (SDN) controllers. And routing protocols compute the forwarding tables that are used to forward packets through the network. The final component of the network layer is a facility to report errors in datagrams and respond to requests for certain network-layer information. Here, we have the ICMP (Internet Control Message Protocol) protocol.

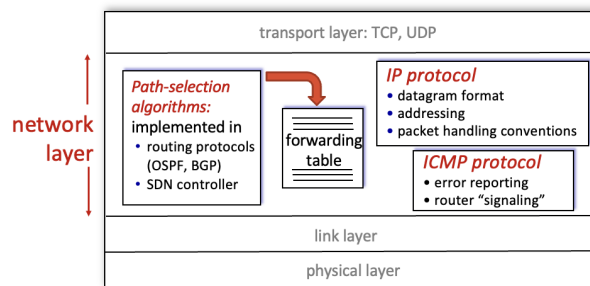


Figure 8: A look inside the Internet's network layer

IP Datagram Format An IP datagram is the basic unit of data transmitted over the Internet, encapsulating both the payload and necessary routing information. It consists of two main sections: the header and the data. The header typically includes fields such as the source IP address, destination IP address, protocol, time to live (TTL), and checksum, which are used for addressing and error detection. The version field specifies whether the datagram is using IPv4 or IPv6. The payload, or data section, contains the actual information being transmitted. The maximum size of an IP datagram is determined by the Maximum Transmission Unit (MTU) of the underlying network, often 1500 bytes for Ethernet. If the datagram exceeds this size, it can be fragmented into smaller pieces.

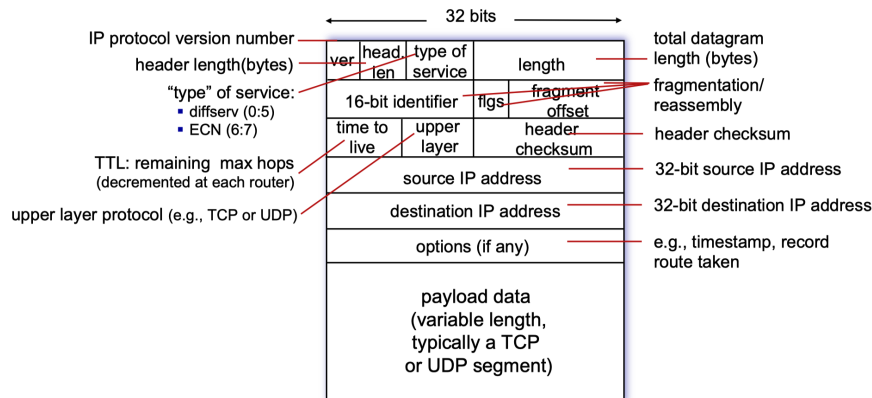


Figure 9: IPv4 datagram format

IPv4 Addressing Because every host and router is capable of sending and receiving IP datagrams, IP requires each host and router interface to have its own IP address. Thus, an IP address is technically associated with an interface, rather than with the host or router containing that interface. Each IP address is 32 bits long (equivalently, 4 bytes), and there are thus a total of 2^{32} possible IP addresses. The 32-bit identifier associated with each host or router interface. Interface is connection between host/router and physical link. Router's typically have multiple interfaces, host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11). Figure 10 provides an example of IP addressing and interfaces.

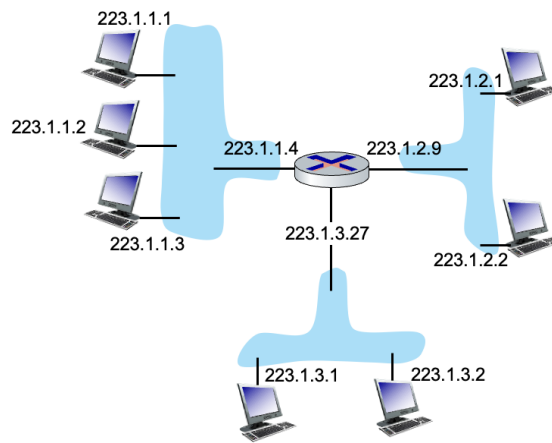


Figure 10: Interface addresses and subnets

There are about 4 billion possible IP addresses. These addresses are typically written in so-called dotted-decimal notation, in which each byte of the address is written in its decimal form and is separated by a period (dot) from other bytes in the address. For example, consider the IP address 223.1.1.1. The 223 is the decimal equivalent of the first 8 bits of the address; the 32 is the decimal equivalent of the second 8 bits of the address, and so on. Thus, the address 223.1.1.1 in binary notation is 11011111 00000001 00000001 00000001.

dotted-decimal IP address notation:
 223.1.1.1 = 11011111 00000001 00000001 00000001
 └───┬───┬───┬───┘
 223 1 1 1

Figure 11: Dotted-decimal IP address notation

Each interface on every host and router in the global Internet must have an IP address that is globally unique (except for interfaces behind NATs). A portion of an interface's IP address will be determined by the subnet to which it is connected.

The table below summarizes the key characteristics of the different IPv4 address classes, including their address ranges, default subnet masks, and typical use cases.

Class	Range	First Octet	Default Subnet Mask	Use Case
A	0.0.0.0 to 127.255.255.255	0 to 127	255.0.0.0	Large networks (e.g., ISPs)
B	128.0.0.0 to 191.255.255.255	128 to 191	255.255.0.0	Medium to large networks
C	192.0.0.0 to 223.255.255.255	192 to 223	255.255.255.0	Small networks
D (Multicast)	224.0.0.0 to 239.255.255.255	224 to 239	N/A	Multicast communication
E (Experimental)	240.0.0.0 to 255.255.255.255	240 to 255	N/A	Experimental use

Table 1: IPv4 Address Classes

In practice, the introduction of **CIDR (Classless Inter-Domain Routing)** replaced the strict class-based system, allowing for more flexible and efficient IP address allocation.

Subnets An IP subnet (short for “subnetwork”) is a logical subdivision of an IP network. Subnets help organize large networks into smaller, manageable segments, improving both network performance and security. Each subnet operates as its own independent network, yet all subnets within the same

larger network can communicate through routers.

In IPv4, subnetting is accomplished by extending the default subnet mask of an IP address. This allows dividing a network into smaller parts, known as subnets, by “borrowing” bits from the host portion of the address to create more network addresses. Each subnet can support a certain number of hosts, which is determined by the number of bits reserved for the host portion of the address.

Subnetting enables more efficient use of IP address space, improves traffic routing by reducing the size of broadcast domains, and provides better control over network access and security. In modern networks, subnetting plays a vital role in optimizing communication within organizations of various sizes.

IP Subnetting Example Given the following information:

- **IP Address:** 192.168.10.0
- **Subnet Mask:** 255.255.255.192 (/26 in CIDR notation)

Step 1: Convert the Subnet Mask to Binary

The subnet mask 255.255.255.192 is converted to binary as follows:

$$255 = 11111111, 255 = 11111111, 255 = 11111111, 192 = 11000000$$

Thus, the subnet mask in binary is:

$$11111111.11111111.11111111.11000000$$

Step 2: Determine the Number of Subnets and Hosts

The subnet mask /26 means 26 bits are used for the network portion, leaving 6 bits for the host portion.

- **Number of Subnets:** The number of subnets is determined by the number of bits borrowed for subnetting ($26 - 24 = 2$ bits for Class C). Thus, the number of subnets is:

$$2^2 = 4 \text{ subnets}$$

- **Hosts per Subnet:** The number of hosts per subnet is determined by the number of bits remaining for hosts (6 bits). Thus, the number of hosts per subnet is:

$$2^6 - 2 = 64 - 2 = 62 \text{ hosts}$$

(subtracting 2 for the network and broadcast addresses).

Step 3: Determine the Subnet Ranges

The block size is 64 (since $256 - 192 = 64$). The subnets are:

- **Subnet 1:** 192.168.10.0/26
 - Network address: 192.168.10.0

- First usable host: 192.168.10.1
- Last usable host: 192.168.10.62
- Broadcast address: 192.168.10.63
- **Subnet 2:** 192.168.10.64/26
 - Network address: 192.168.10.64
 - First usable host: 192.168.10.65
 - Last usable host: 192.168.10.126
 - Broadcast address: 192.168.10.127
- **Subnet 3:** 192.168.10.128/26
 - Network address: 192.168.10.128
 - First usable host: 192.168.10.129
 - Last usable host: 192.168.10.190
 - Broadcast address: 192.168.10.191
- **Subnet 4:** 192.168.10.192/26
 - Network address: 192.168.10.192
 - First usable host: 192.168.10.193
 - Last usable host: 192.168.10.254
 - Broadcast address: 192.168.10.255

NAT: Network Address Translation Assume that you have 15 PCs, one smartphone, two ipads and all of them need to work at the same time, then you need to get each of them an IP address accessible to the Internet. But due to a lack of IPv4 IP address space, it is hard to handle the massive number of devices we use every day. NAT proposed in 1994, has become a popular and necessary tool in the face of IPv4 address exhaustion by representing all internal devices as a whole with the same public address available. It also provides safety. For example, hackers from outside cannot directly attack the internal network while the internal information cannot access the outside world casually. The idea of network address translation or NAT is all devices in a local network share just one IPv4 address as far as the outside world is concerned.

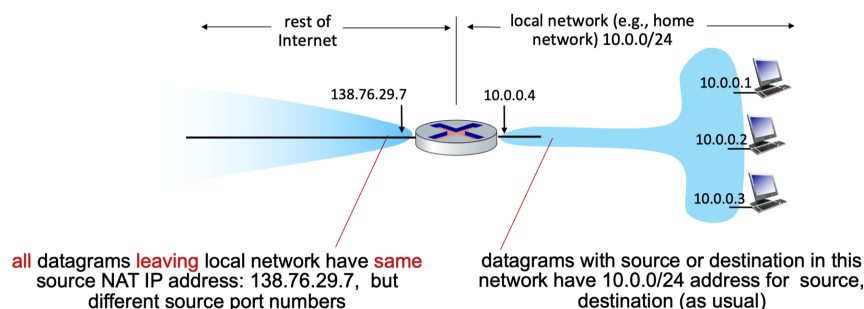
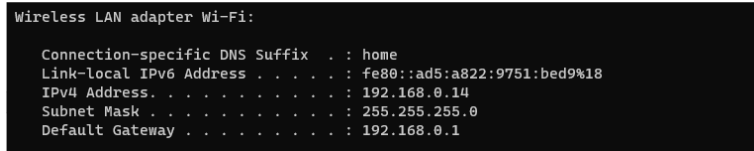


Figure 12: NAT

Figure 12 shows the operation of a NAT-enabled router. Addressing within the home network is exactly as we have seen above—all four interfaces in the home network have the same subnet address of 10.0.0/24. These datagrams with source or destination in this network have 10.0.0/24 address for

source, destination (as usual). All datagrams leaving the local network have the same source NAT IP address: 138.76.29.7, but different source port numbers. We use NAT translation table to map private IP address and public IP address. For example, public IP add 138.76.29.7, 5007 will be mapped to 10.0.0.1, 3345.

IPv6 A prime motivation for IPv6 was the realization that the 32-bit IP address space was beginning to be used up, with new subnets and IP nodes being attached to the Internet (and being allocated unique IP addresses) at a breathtaking rate. To respond to this need for a large IP address space, a new IP protocol, IPv6, was developed. Additionally, IPv6 can speed processing and forward with 40-byte fixed-length header and enable different network-layer treatments of ‘flows’. An example of a network interface with both IPv4 and IPv6 addresses is shown in Figure 13.



```
Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix . : home
Link-local IPv6 Address . . . . . : fe80::ad5:a822:9751:bed9%18
IPv4 Address. . . . . : 192.168.0.14
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.0.1
```

Figure 13: IPv6 Example

2 Attacks on Networks

We define attacks on networks as events or sequences of actions that potentially violate one or more security goals. For instance, a DDoS attack on a server compromises the security goal of availability. This highlights a critical distinction from simpler software security scenarios where the input to systems, such as web servers or mail servers, originates remotely, and there’s initially no control over who can send input. This remote accessibility allows attackers to send any type or amount of traffic to a network, increasing the vulnerability to attacks. Moreover, once an attacker successfully infiltrates a network, they gain ample opportunities to explore, breach, and exploit the system, further compounding the security challenges.

Passive attacks are in the nature of eavesdropping on, or monitoring of, transmissions. The goal of the attacker is to obtain information that is being transmitted. Two types of passive attacks are the release of message contents and traffic analysis. Passive attacks are very difficult to detect because they do not involve any alteration of the data. **Active attacks** involve some modification of the data stream or the creation of a false stream and can be subdivided into four categories: replay, masquerade, modification of messages, and denial of service. A masquerade takes place when one entity pretends to be a different entity. For example, authentication sequences can be captured and replayed after a valid authentication sequence has taken place, thus enabling an authorized entity with few privileges to obtain extra privileges by impersonating an entity that has those privileges.

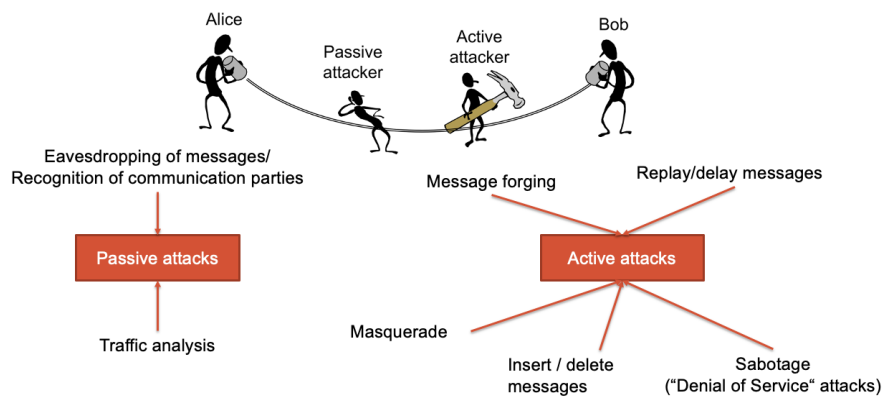


Figure 14: Attacks on Communication Networks

There are ways to compromise security goals in networks:

- **Masquerade:** An entity claims to be another entity (also called "Impersonation")
- **Eavesdropping:** An entity reads the information it is not intended to read. For example, the attacker can perform a man-in-the-middle attack and capture your packet's information. Sometimes it can be login information if the connection is not secure (HTTPS)
- **Loss or modification of (transmitted) information:** Data is being altered or destroyed. For example, after performing a man-in-the-middle attack, the attacker can intercept the packet, change the data and forward it to the server. Let's think about you're changing your account's password; the attacker could intercept your request and change your password to his/her password then forward the packet to the server
- **Forgery of information:** an entity creates new information in the name of another entity. For example, after gaining your credential and accessing your account, he/she changed your address to his/her address
- **Denial of accountability:** An entity falsely denies its participation in a communication act. For instance, because the attacker used your credential to perform bad actions; if something illegal involves, he/she can deny, and you might need to prove you did not do it by somehow
- **Sabotage/Denial of Service:** Any action that aims to reduce the availability and / or correct functioning of services or systems. An ideal example here is a DDoS attack on a remote server
- **Authorization Violation:** An entity uses a service or resources it is not intended to use. Going back to the example that the attacker takes over your account, the second his/her accesses your account, it's an authorization violation

3 Network Security - Protocols

In the realm of network security, cryptographic protocols are employed across different layers to enhance data protection. Here are some common cryptographic protocols by layer. At the link layer, protocols such as WPA3 and WPA2 are used to secure routers, though earlier versions like WPA and WEP are now considered compromised and insecure. The network layer employs IPSec and IKEv2 to safeguard communications, ensuring both integrity and confidentiality. For the transport layer, TLS is utilized for TCP connections and DTLS for UDP, providing robust security measures for transmitting data. At the application layer, security mechanisms include OpenID and OAuth for authentication and authorization, along with DNSSEC to secure the Domain Name System. In professional settings, the most commonly encountered protocols are TLS, WPA2/WPA3, OpenID, OAuth, and IPSec.

3.1 Application Layer: OAuth and OpenID

At the top of the TCP/IP model is the application layer, where security protocols such as OAuth and OpenID are crucial. OAuth is a standard for authorization that allows users of one internet service to grant controlled access to their account to another service, reminiscent of Kerberos which uses tickets to authorize access. For instance, if you have authenticated your Google account, you can use OAuth to grant access specifically to your email information without sharing other details. The key benefit of OAuth is that it eliminates the need to repeatedly enter usernames and passwords; instead, users register with one service provider and utilize authorization tokens for other services. Designed to work seamlessly with HTTP/S, OAuth supports complex protocol flows with several standardized options.

OpenID, on the other hand, focuses on authentication. Using the Google example, you could authenticate on another website using your Google ID, leveraging OAuth flows. Nowadays, many accounts are linked across various services; for example, a single Facebook or Google account can be used to access numerous other services, reducing the need to create multiple accounts and store additional information on different websites. Despite its complexity, in cloud services, protocols like OpenID have become the standard due to their efficiency and the security they offer.

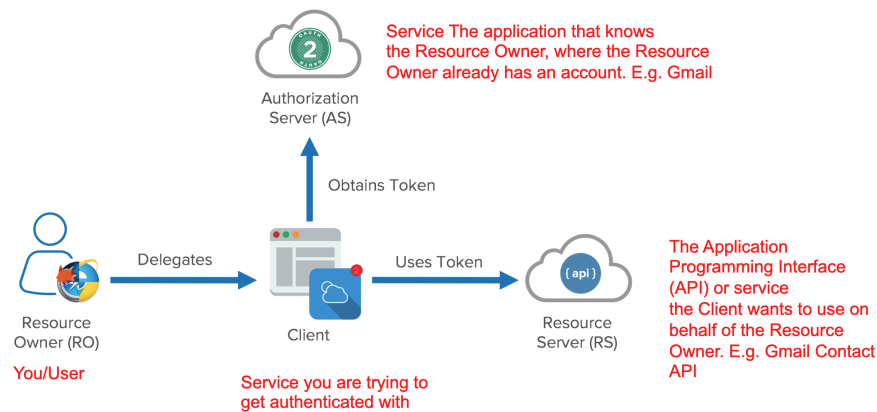


Figure 15: OAuth

As shown in Figure 15, the actors in OAuth flows are as follows:

- **Resource Owner:** owns the data in the resource server. For example, I'm the Resource Owner of my Facebook profile
- **The API** which stores data the application wants to access
- **Client:** the application that wants to access your data
- **Authorization Server:** The main engine of OAuth

3.2 Transport Layer: Transport layer security (TLS)

One of the most widely used security services is Transport Layer Security (TSL); the latest version is 1.3. TLS is an Internet standard that evolved from a commercial protocol known as Secure Sockets Layer (SSL). However, SSL3 has been phased out (marginal usage). Originally developed by Netscape, which later became Mozilla, Secure Socket Layer (SSL) underwent significant evolution before becoming robust. The first version, SSLv1, was found to be very flawed and never saw real deployment. SSLv2, although deployed, also had significant flaws. It was not until SSLv3 that a robust version emerged, which was later standardized by the Internet Engineering Task Force

(IETF) as Transport Layer Security (TLS). Although SSL implementations are still around, it has been deprecated by IETF and is disabled by most corporations offering TLS software. TLS is a general-purpose service implemented as a set of protocols that rely on TCP. At this level, there are two implementation choices. For full generality, TLS could be provided as part of the underlying protocol suite and therefore be transparent to applications. Alternatively, TLS can be embedded in specific packages. For example, most browsers come equipped with TLS, and most Web servers have implemented the protocol.

The original purpose of SSL/TLS was to secure HTTP at the application layer, where messages were previously unencrypted. To avoid altering the application layer, the design goal is to create a generic security layer (layer 4.5) between TCP and the application layer. Because TLS is a general-purpose service implemented as a set of protocols that rely on TCP, TLS should make use of TCP underneath and reuse of 'socket abstraction' allowing applications to interface with TLS sockets just like TCP sockets. Another design goal is the use of encryption, integrity and origin authentication as transparent to the application as possible. Demonstrating HTTP's vulnerabilities, tools like Entercap can perform man-in-the-middle attacks within the same network, capturing images and videos viewed by users. Why not placing TLS on the application layer? It is because every application layer protocol would have to be extended to use it. Back at that time, the application layer was already changed and extended. Given that these protocols were often text-based and already complex, placing TLS at the transport layer, which uses bytes, was more straightforward and efficient.

3.2.1 Socket (revisiting - bit more detailed)

An application requests network communication from the kernel by defining its requirements, such as reliable, stream-based communication. For example, if the application wants to communicate with other applications, it uses socket, which is an object into which the application can write, and from which it can read—very much like a file descriptor. Socket is abstraction over source IP, destination IP, source port, destination port, and protocol used. Raw socket is a socket without any layer-specific formatting done by the kernel—all left to application. The kernel handles the sending and receiving of data over the network. For reliable, connection-oriented communication, TCP is used, which is referred to as 'streaming communication.' Conversely, 'datagram communication' via UDP is used for unreliable, message-oriented communication.

Sockets are created with system calls, using a standardised API. TLS sockets are implemented in libraries that handle the setup of ciphers, MACs, and other security protocols. These libraries collaborate with the kernel to create the socket, which is then provided to the application. The TLS socket provided behaves similarly to a standard TCP socket, allowing secure and encrypted communication without altering the familiar TCP socket interface used by the application.

In Python, establishing a secure connection starts with opening a TCP socket, which then needs to be wrapped to become a TLS socket for encrypted communication. Here's how you typically open a TCP connection:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((HOST, PORT))
```

To secure this connection, you use the `.wrap_socket` method from the `ssl` library to wrap the TCP socket into a TLS socket, which is capable of encrypted communication. This process might look like this:

```
t = ssl.wrap_socket(s, ...)
```

In the ..., security parameters are defined, potentially including settings like Diffie-Hellman for key exchange. Given the complexities of security and cryptography, it's crucial to adhere to secure default settings provided by libraries to avoid vulnerabilities. The common design involves the TLS socket wrapping around the normal socket, with many parameters available for this wrapping process. Using safe defaults is vital to ensure robust security.

3.2.2 Conceptual View of TLS

TLS relies on the presence of an X.509 Public Key Infrastructure (PKI) for authenticating parties, each of which must possess an X.509 certificate. While other variants like pre-shared key exist, they are relatively rare.

TLS 1.3 always uses authenticated Diffie-Hellman, ensuring forward security. In contrast, TLS 1.2 strongly recommended this approach but also supported traditional hybrid cryptography, where the key is jointly created by the client and server through a more complex key creation process.

Authentication in TLS can be mutual; however, it is more common for only the server to authenticate to the client. On the application layer, client applications often authenticate using methods such as passwords.

3.2.3 Conceptual Protocol Flow

We show the idea of TLS for server-only authentication, with Diffie-Hellman. Let the actors be client C and server S . Secret Diffie-Hellman values are c and s , respectively:

```
C → S : NC
S → C : NS, CertS, gs, SigS(NC, NS, gs)

C → S : gc, MACKC,S(X)
(Let X := NC, NS, CertS, gs, gc, SigS(NC, NS, gs))

S → C : MACKC,S(Y)
(Let Y := (X, MACKC,S(X)))
```

The first half of the TLS protocol flow is mainly the negotiation of the cryptography and declaration of **nonces** that will be used for **freshness**:

- $C \rightarrow S : N_C$
Client sends nonce to identify new session
- $S \rightarrow C : N_S, \text{Cert}_S, g^s, \text{Sig}_S(N_C, N_S, g^s)$
Server picks nonce of its own to refer to new session; sends certificate; picks Diffie-Hellman value; signs everything

The second half is the actual **authentication and key establishment**. The pattern of 'two messages for negotiation, two for AKE' is quite common in protocols:

- $C \rightarrow S : g^c, \text{MAC}_{K_{C,S}}(X)$
(Let $X := N_C, N_S, \text{Cert}_S, g^s, g^c, \text{Sig}_S(N_C, N_S, g^s)$)
Client can authenticate S thanks to the certificate and the signed nonce. It can derive the symmetric key and use MACs from this point on. Client sends Diffie-Hellman value of its own

- $S \rightarrow C : MAC_{K_{C,S}}(Y)$
 $(Let Y := (X, MAC_{K_{C,S}}(X)))$

Correctness of Diffie-Hellman can be checked via MAC. Server has symmetric key now, too. Server creates final MAC on everything. When receiving the final MAC, client can be assured that no parameters have been tampered with.

3.2.4 Negotiation in TLS

TLS negotiates concrete ciphers, hash functions, and MACs to be used between the communicating parties, a practice that is common across many cryptographic protocols. The TLS standard includes codepoints for supported mechanisms, allowing the client to propose a suite of mechanisms from which the server either selects its preference or rejects the connection if none are suitable.

Additionally, TLS supports various extensions that enhance its functionality. One key extension is the Server Name Indication, which specifies the domain name intended for the connection. This extension also helps in identifying the specific protocol that is being protected. Other useful mechanisms supported by TLS include Certificate Transparency, which adds a layer of security by ensuring certificates are publicly logged, and indications of application-layer protocols or unusual cryptographic approaches, enhancing both transparency and security.

3.2.5 TLS Subprotocols

TLS is message-based and the subprotocols define message exchanges. TLS is designed to make use of TCP to provide a reliable end-to-end secure service. TLS is not a single protocol but rather two layers of protocols. The TLS Record Protocol provides basic security services to various higher layer protocols. In particular, the Hypertext Transfer Protocol (HTTP) or application data, can operate on top of TLS. Three higher-layer protocols are defined as part of TLS:

1. **Handshake Protocol:** This protocol (named for its function, not as a category) facilitates the negotiation of session details such as cryptographic algorithms and session keys.
2. **Change Cipher Spec Protocol:** This signals transitions in the cryptographic methods being used.
3. **Alert Protocol:** Used to signal errors or issues during the TLS process.

Additionally, TLS manages Application Data, which refers to the secured payloads carried over the network. These protocols play crucial roles in managing TLS exchanges, ensuring secure communications over networks.



Figure 16: TLS Subprotocols

3.2.6 TCP Handshake Protocol

Our ‘conceptual protocol flow’ is really the Handshake Protocol. Its messages map to our abstract notation. The most important ones are:

- ClientHello: Version number, nonce, cipher suites

- ServerHello: Version number, nonce, cipher suites, session ID
- ServerCertificate: certificate
- ServerKeyExchange: Diffie-Hellman parameter
- ClientKeyExchange: Diffie-Hellman value
- Finished: MACs

Messages of the Change CipherSpec protocol may appear before or after messages of the Handshake Protocol; they signal that the next messages use (different) encryption.

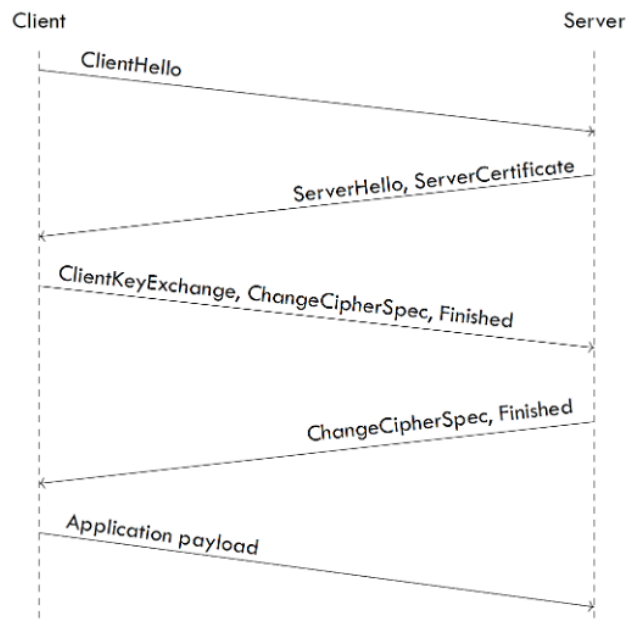


Figure 17: Protocol Flow of TLS 1.2

As shown in Figure 17, client first established security capabilities, including protocol version, session ID, cipher suite, compression method, and initial random numbers. Server may send certificate, key exchange, and request certificate. Server signals end of hello message phase. Client sends certificate if requested. Client sends key exchange. Client may send certificate verification. Server changes cipher suite and finish handshake protocol. Client then sends the application payload.

3.2.7 TLS 1.3

TLS 1.2 was already the result of many improvements to TLS 1.0 and then TLS 1.1. However, it still contains weaknesses due to its use of relatively old cryptography, MAC-then-encrypt instead of Encrypt-then-MAC, use of compression. From about 2008, serious pressure on the protocol - redesign was needed. Table 2 provides a feature summary of different TLS versions.

Major Differences in TLS 1.3 TLS 1.2 permitted the use of a null cipher, which provided authentication without encryption; however, this is no longer allowed. In contrast, TLS 1.3 mandates the use of AEAD (Authenticated Encryption with Associated Data) ciphers only, such as AES-GCM and ChaCha20-Poly1305, enhancing security by ensuring that both confidentiality and integrity are maintained. Furthermore, TLS 1.3 prioritizes Diffie-Hellman as the default key exchange mechanism, except in a specific mode that uses a pre-shared key. Encryption begins immediately following the

Feature	TLS 1.0	TLS 1.1	TLS 1.2	TLS 1.3
Introduced	1999	2006	2008	2018
Supported Hash Algorithms	MD5, SHA-1	MD5, SHA-1	SHA-1, SHA-256, SHA-384	SHA-256, SHA-384
Handshake Security	Vulnerable to downgrade attacks	Improved handshake integrity checks	HMAC-based authentication, flexible cipher suites	Simplified handshake, forward secrecy by default
Cipher Suites	Supports weak ciphers like DES, RC4	RC4 deprecated, still supports CBC-mode ciphers	Secure ciphers (AES-GCM, AES-CCM), supports Authenticated Encryption (AE)	Only supports secure AE ciphers (AES-GCM, ChaCha20-Poly1305)
Key Exchange Methods	RSA, DH	RSA, DH, DHE	RSA, DH, DHE, ECDHE	Only ECDHE and PSK (Pre-Shared Key)
Renegotiation	Vulnerable to renegotiation attacks	Renegotiation fixed	Supports secure renegotiation	Renegotiation eliminated
Forward Secrecy	Not required	Optional	Optional but widely supported	Mandatory (by ECDHE)
Session Resumption	Session IDs only	Session IDs only	Session IDs and session tickets	0-RTT for session resumption with tickets
Round Trips for Handshake	2 RTs	2 RTs	2 RTs	1 RT (0-RTT option for resumption)
Deprecation Status	Deprecated	Deprecated	Widely used, but moving toward deprecation	Actively recommended

Table 2: Comparison of TLS Versions (TLS 1.0, TLS 1.1, TLS 1.2, TLS 1.3)

ServerHello message. This means that certificates are encrypted and integrity-protected, as are the Diffie-Hellman values shared during the KeyShare process. A significant efficiency improvement in TLS 1.3 is the 1-RTT feature, which allows the client to send application payload data after just one round-trip time. This enhancement drastically reduces the time required to establish a secure connection, speeding up secure communications.

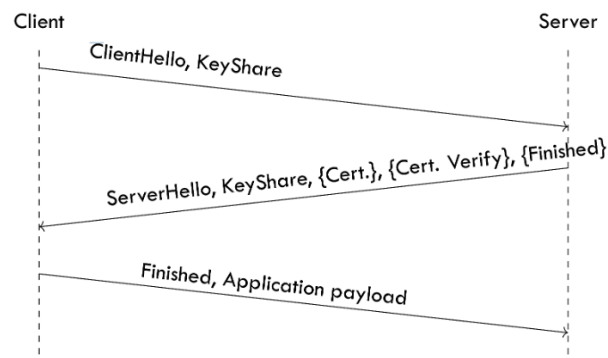


Figure 18: Protocol Flow of TLS 1.3

Figure 19 is a detailed diagram explaining TLS 1.3 handshake with 1 round trip time (1-RTT).

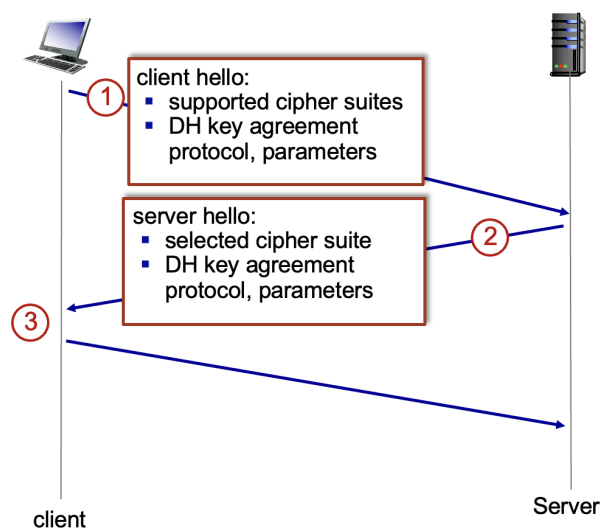


Figure 19: TLS 1.3 Handshake: 1-RTT

1. First, client sends TLS hello msg that is used to guess keys agreement protocol, parameters and indicate cipher suites it supports
2. Then, server sends TLS hello msg chooses key agreement protocol, parameters, cipher suite, server-signed certificate
3. Finally, client checks server certificate, generates key, and can now make application request (e.g., HTTPS GET)

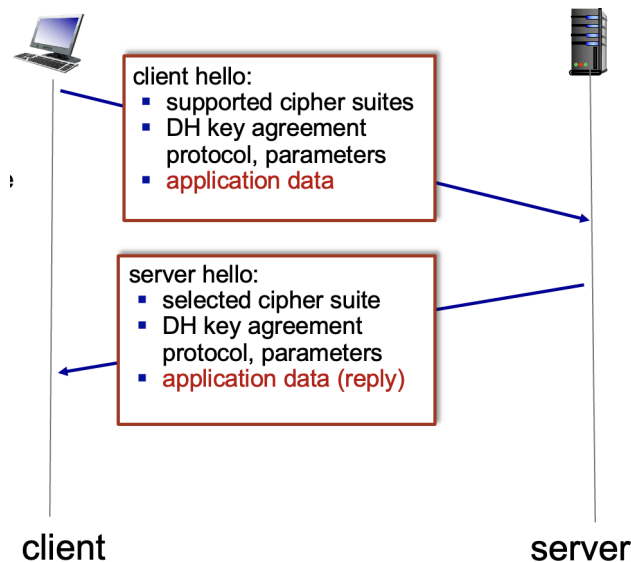


Figure 20: 0-RTT in TLS 1.3

TLS 1.2 introduced two session resumption features aimed at reducing the time required to re-establish connections using just one round-trip time (RTT). The first feature is the Session Identifier, which allows servers to quickly look up a previously used key via a unique session identifier. The second feature is the Session Ticket, where the client holds a token that can be sent to the server to resume the connection efficiently.

TLS 1.3 deprecates these methods in favor of a 'preshared key' mode, which operates in a manner conceptually similar to the session ticket. This version of TLS also integrates the use of the "Early Data" extension to facilitate 0-RTT data transmission, a design choice aimed at improving performance but not without its security trade-offs. The problem with 0-RTT is the use of **session key is not forward-secure**; if the session key is compromised, an attacker could access all encrypted data. Additionally, there is **no replay protection** with 0-RTT, which means data could potentially be resent by an attacker. As a result, 0-RTT should only be used for idempotent messages at the application layer, such as HTTP GET requests. The problem is that it's unknown if operators will implement this correctly. 0-round trip time (0-RTT) in TLS 1.3 is shown in Figure 20.

Read more about TLS 1.3 here:

<https://blog.cloudflare.com/tls-1-3-overview-and-q-and-a/>

3.3 Internet Layer: IPsec

IP security (IPsec) provides security at Network Layer. IP-level security encompasses three functional areas: authentication, confidentiality, and key management. The authentication mechanism assures that a received packet was, in fact, transmitted by the party identified as the source in the packet header. In addition, this mechanism assures that the packet has not been altered in transit. The confidentiality facility enables communicating nodes to encrypt messages to prevent eavesdropping by third parties. The key management facility is concerned with the secure exchange of keys. IPsec embeds its protective mechanisms as IP payloads.

The advantage of using IPsec is that it is transparent to end users and applications and provides a general-purpose solution. Furthermore, IPsec includes a filtering capability so that only selected traffic need incur the overhead of IPsec processing. For example, an enterprise can run a secure,

private IP network by disallowing links to untrusted sites, encrypting packets that leave the premises, and authenticating packets that enter the premises. By implementing security at the IP level, an organization can ensure secure networking not only for applications that have security mechanisms but also for the many security-ignorant applications.

The differences between TLS and IPSec are notable in several aspects. Unlike TLS, which does not protect any part of the TCP packet information such as port numbers or segment numbers, IPSec provides authentication and integrity protection for the actual IP packets. Additionally, TLS operates in user space, allowing it to be implemented and managed at the application level. In contrast, IPSec must be run in kernel space, meaning its configuration is restricted to system administrators. This distinction highlights the deeper integration of IPSec with the operating system and the more flexible application-level deployment of TLS.

3.3.1 IPv4 Packet Format (revisiting)

Ver.	IHL	TOS	Length	
IP Identification			Flags	Fragment Offset
TTL	Protocol		IP Checksum	
Source Address				
Destination Address				
IP Options (if any)				
TCP / UDP / ... Payload				

Figure 21: IPv4 Packet Format

When an entity receives an IP packet, it faces several uncertainties: there is no assurance of the data's origin due to the possibility of IP spoofing by attackers, no means to verify data integrity, making it impossible to confirm if the data has been altered in transit, and no confidentiality, as IP packets are not encrypted. The idea of IPSec is to configure the protection for IP packets between two IP address/address ranges - done with a policy.

3.3.2 Applications of IPSec

IPsec provides the capability to secure communications across a LAN, across private and public WANs, and across the Internet. Examples of its use include:

- **Secure branch office connectivity over the Internet:** A company can build a secure virtual private network over the Internet or over a public WAN. This enables a business to rely heavily on the Internet and reduce its need for private networks, saving costs and network management overhead.
- **Secure remote access over the Internet:** An end user whose system is equipped with IP security protocols can make a local call to an Internet Service Provider (ISP) and gain secure access to a company network. This reduces the cost of toll charges for traveling employees and telecommuters.
- **Establishing extranet and intranet connectivity with partners:** IPsec can be used to secure communication with other organizations, ensuring authentication and confidentiality and providing a key exchange mechanism.
- **Enhancing electronic commerce security:** Even though some Web and electronic commerce applications have built-in security protocols, the use of IPsec enhances that security. IPsec

guarantees that all traffic designated by the network administrator is both encrypted and authenticated, adding an additional layer of security to whatever is provided at the application layer.

The principal feature of IPsec that enables it to support these varied applications is that it can encrypt and/or authenticate all traffic at the IP level. Thus, all distributed applications (including remote logon, client/server, email, file transfer, Web access, and so on) can be secured.

3.3.3 IPsec Services

IPsec provides security services at the IP layer by enabling a system to select required security protocols, determine the algorithm(s) to use for the service(s), and put in place any cryptographic keys required to provide the requested services. Two protocols are used to provide security: an authentication protocol designated by the header of the protocol, **Authentication Header (AH)**; and a combined encryption/authentication protocol designated by the format of the packet for that protocol, **Encapsulating Security Payload (ESP)**. RFC 4301 lists the following services:

- Access control
- Connectionless integrity
- Data origin authentication
- Rejection of replayed packets (a form of partial sequence integrity)
- Confidentiality (encryption)
- Limited traffic flow confidentiality

3.3.4 IPsec Architecture Overview

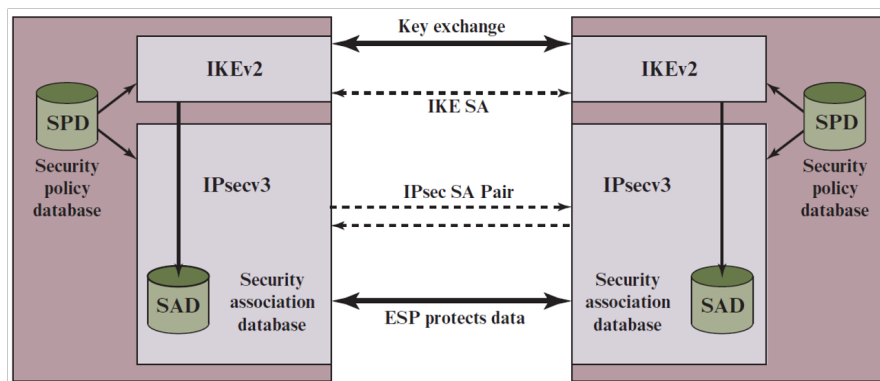


Figure 22: IPsec Architecture

Fundamental to the operation of IPsec is the concept of a security policy applied to each IP packet that transits from a source to a destination. IPsec policy is determined primarily by the interaction of two databases, the security association database (SAD) and the security policy database (SPD). Figure 22 illustrates the relevant relationships.

3.3.5 Components of IPsec

These components of IPsec are:

- i) Security Association Database (SAD)
- ii) Security Policy Database (SPD)
- iii) ESP: Encapsulating Security Payloads with authentication, integrity, and confidentiality
- iv) Authentication Header (AH) with authentication and integrity for IP header (not payload)
- v) IKEv2: Internet Key Exchange with AKE protocol
- vi) Two different modes: tunnel and transport

IPsec is highly complex. Thus, some functionalities are even duplicated.

i) Security Association (SA) A key concept that appears in both the authentication and confidentiality mechanisms for IP is the security association (SA). Security Association describes the concrete way how incoming/outgoing packets are going to be protected. An SA is a negotiated outcome, with the negotiation based on the definition of the Security Policy. An association is a one-way logical connection between a sender and a receiver that affords security services to the traffic carried on it. If a peer relationship is needed for two-way secure exchange, then two security associations are required. In each IPsec implementation, there is a nominal Security Association Database that defines the parameters associated with each SA. In short, SA is stored in the security association database. SAs can be combined in a number of ways to yield the desired user configuration. Figure 23 is an illustration of an SA between two hosts.

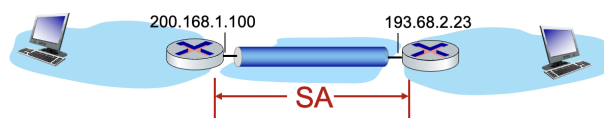


Figure 23: SA between two hosts

ii) Security Policy Database The means by which IP traffic is related to specific SAs (or no SA in the case of traffic allowed to bypass IPsec) is the nominal Security Policy Database (SPD). In its simplest form, an SPD contains entries, each of which defines a subset of IP traffic and points to an SA for that traffic. In more complex environments, there may be multiple entries that potentially relate to a single SA or multiple SAs associated with a single SPD entry. Each SPD entry is defined by a set of IP and upper-layer protocol field values, called selectors. In short, selectors define IP flow. In effect, these selectors are used to filter outgoing traffic in order to map it into a particular SA. Outbound processing obeys the following general sequence for each IP packet.

1. Compare the values of the appropriate fields in the packet (the selector fields) against the SPD to find a matching SPD entry, which will point to zero or more SAs.
2. Determine the SA if any for this packet and its associated SPI.
3. Do the required IPsec processing (i.e., AH or ESP processing).

In each IPsec implementation, there is a nominal Security Association Database that defines the parameters associated with each SA. A security association is normally defined by the following parameters in an SAD entry.

- **Security Parameter Index:** A 32-bit value selected by the receiving end of an SA to uniquely identify the SA. In an SAD entry for an outbound SA, the SPI is used to construct the packet's AH or ESP header. In an SAD entry for an inbound SA, the SPI is used to map traffic to the appropriate SA.
- **Sequence Number Counter:** A 32-bit value used to generate the Sequence Number field in AH or ESP headers (required for all implementations).
- **Sequence Counter Overflow:** A flag indicating whether overflow of the Sequence Number Counter should generate an auditable event and prevent further transmission of packets on this SA (required for all implementations).
- **Anti-Replay Window:** Used to determine whether an inbound AH or ESP packet is a replay (required for all implementations).
- **AH Information:** Authentication algorithm, keys, key lifetimes, and related parameters being used with AH (required for AH implementations).
- **ESP Information:** Encryption and authentication algorithm, keys, initialization values, key lifetimes, and related parameters being used with ESP (required for ESP implementations).
- **Lifetime of this Security Association:** A time interval or byte count after which an SA must be replaced with a new SA (and new SPI) or terminated, plus an indication of which of these actions should occur (required for all implementations).
- **IPsec Protocol Mode:** Tunnel, transport, or wildcard.
- **Path MTU:** Any observed path maximum transmission unit (maximum size of a packet that can be transmitted without fragmentation) and aging variables (required for all implementations).

The key management mechanism that is used to distribute keys is coupled to the authentication and privacy mechanisms only by way of the Security Parameters Index (SPI). Hence, authentication and privacy have been specified independent of any specific key management mechanism.

IPsec provides the user with considerable flexibility in the way in which IPsec services are applied to IP traffic. As we will see later, SAs can be combined in a number of ways to yield the desired user configuration. Furthermore, IPsec provides a high degree of granularity in discriminating between traffic that is afforded IPsec protection and traffic that is allowed to bypass IPsec, as in the former case relating IP traffic to specific SAs.

Protocol	Local IP	Port	Remote IP	Port	Action	Comment
UDP	1.2.3.101	500	*	500	BYPASS	IKE
ICMP	1.2.3.101	*	*	*	BYPASS	Error messages
*	1.2.3.101	*	1.2.3.0/24	*	PROTECT: ESP intransport-mode	Encrypt intranet traffic
TCP	1.2.3.101	*	1.2.4.10	80	PROTECT: ESP intransport-mode	Encrypt to server
TCP	1.2.3.101	*	1.2.4.10	443	BYPASS	TLS: avoid double encryption
*	1.2.3.101	*	1.2.4.0/24	*	DISCARD	Others in DMZ
*	1.2.3.101	*	*	*	BYPASS	Internet

Figure 24: Host SPD Example

iii) Encapsulating Security Payload (ESP) ESP can be used to provide confidentiality, data origin authentication, connectionless integrity, an anti-replay service (a form of partial sequence integrity), and (limited) traffic flow confidentiality. The set of services provided depends on options selected at the time of Security Association (SA) establishment and on the location of the implementation in a network topology. ESP can work with a variety of encryption and authentication algorithms, including authenticated encryption algorithms such as GCM. ESP can be used stand-alone or combined with Authentication Header.

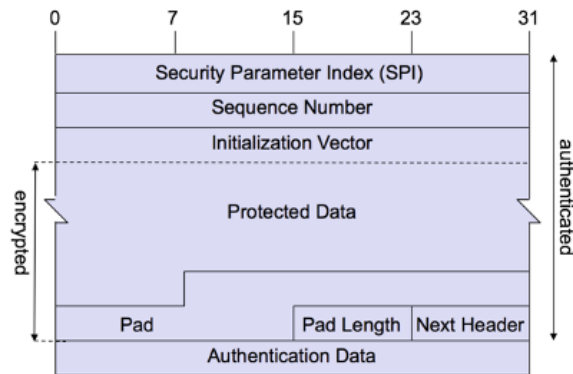


Figure 25: ESP header, protected data, trailer

Looking at the ESP header, we have:

- SPI is used to identify a security association
- Sequence Number (32 bits) is a monotonically increasing counter value; this provides an anti-replay function, as discussed for AH
- An initialization value (IV), or nonce, is present if this is required by the encryption or authenticated encryption algorithm used for ESP
- Payload/Protected Data (variable) is a transport-level segment (transport mode) or IP packet (tunnel mode) that is protected by encryption
- Pad Length (8 bits) indicates the number of pad bytes immediately preceding this field
- Next Header (8 bits) identifies the type of data contained in the payload data field by identifying the first header in that payload (e.g., an extension header in IPv6, or an upper-layer protocol such as TCP)
- Integrity Check Value or authentication data (variable) is a variable-length field (must be an integral number of 32-bit words) that contains the Integrity Check Value computed over the ESP packet minus the Authentication Data field

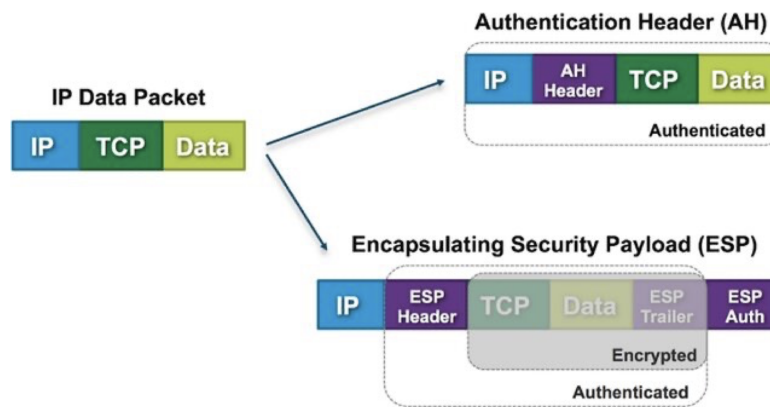


Figure 26: ESP vs. AH

Authentication Header (AH) In IPsec services, two protocols are used to provide security: an authentication protocol designated by the header of the protocol, Authentication Header (AH); and a combined encryption/ authentication protocol designated by the format of the packet for that protocol, Encapsulating Security Payload (ESP). AH authenticates parts of the IP header and the payload. We used AH because parts of the IP header change as the packet is being forwarded (e.g., Time-to-live). When we refer to authenticated parts – there are the ‘normally immutable’ ones: source and destination IP, version, length, etc.

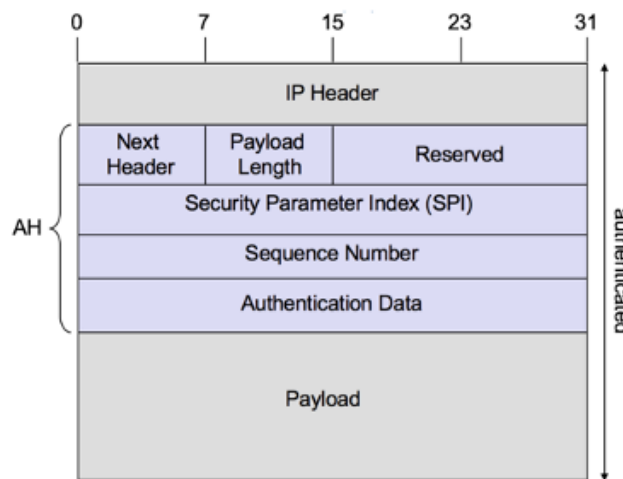


Figure 27: Use of Authentication header.

IKEv2 IKEv2 is designed to provide mutual authentication and key establishment between two parties. It facilitates the negotiation of cryptographic parameters such as ciphers and hash functions. The process results in the creation of a total of four so-called Security Associations (SAs), with one SA allocated for each party and direction. These SAs established via IKEv2 are subsequently utilized by Authentication Header (AH) and/or Encapsulating Security Payload (ESP) protocols. Additionally, the SAs are stored in the Security Association Database, ensuring that the agreed-upon security protocols and keys are maintained and accessible for ongoing and future secure communications.

vi) a. IPsec: Tunnel Mode Tunnel mode provides protection to the entire IP packet. To achieve this, after the AH or ESP fields are added to the IP packet, the entire packet plus security fields is treated as the payload of new outer IP packet with a new outer IP header. The entire original, inner,

packet travels through a tunnel from one point of an IP network to another; no routers along the way are able to examine the inner IP header. Because the original packet is encapsulated, the new, larger packet may have totally different source and destination addresses, adding to the security. Tunnel mode is used when one or both ends of a security association (SA) are a security gateway, such as a firewall or router that implements IPsec. With tunnel mode, a number of hosts on networks behind firewalls may engage in secure communications without implementing IPsec. The unprotected packets generated by such hosts are tunneled through external networks by tunnel mode SAs set up by the IPsec software in the firewall or secure router at the boundary of the local network.

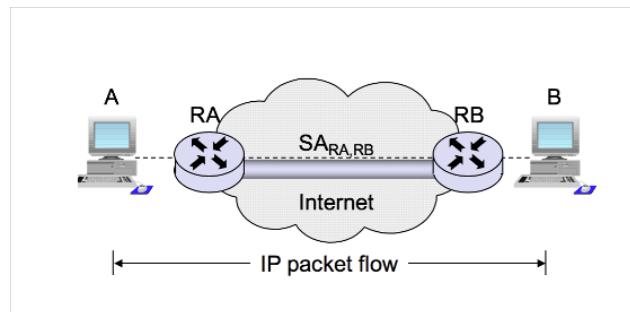


Figure 28: Tunnel Mode

Mode Protocol	Transport	Tunnel
AH	IP AH Data	IP AH IP Data
ESP	IP ESP Data ESP-T	IP ESP IP Data ESP-T
AH-ESP	IP AH ESP Data ESP-T	IP AH ESP IP Data ESP-T

Figure 29: Transport and Tunnel Mode

vii) b. IPsec: Transport Mode Transport mode provides protection primarily for upper-layer protocols. That is, transport mode protection extends to the payload of an IP packet. 1 Examples include a TCP or UDP segment or an ICMP packet, all of which operate directly above IP in a host protocol stack. Typically, transport mode is used for end-to-end communication between two hosts (e.g., a client and a server, or two workstations). When a host runs AH or ESP over IPv4, the payload is the data that normally follow the IP header. For IPv6, the payload is the data that normally follow both the IP header and any IPv6 extensions headers that are present, with the possible exception of the destination options header, which may be included in the protection.

ESP in transport mode encrypts and optionally authenticates the IP payload but not the IP header. AH in transport mode authenticates the IP payload and selected portions of the IP header. Therefore, transport mode operation provides confidentiality for any application that uses it, thus avoiding the need to implement confidentiality in every individual application. One drawback to this mode is that it is possible to do traffic analysis on the transmitted packets.

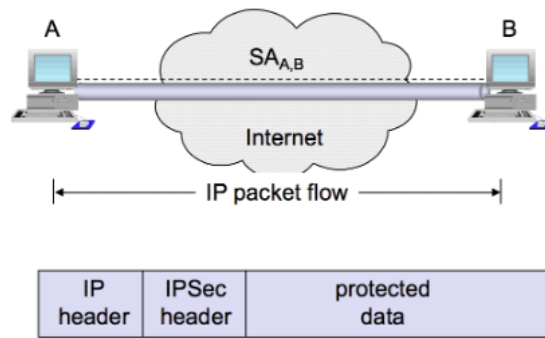


Figure 30: Transport Mode

3.3.6 Virtual Private Network (VPN)

Tunnel mode can be used to implement a secure virtual private network. A virtual private network (VPN) is a private network that is configured within a public network (a carrier's network or the Internet) in order to take advantage of the economies of scale and management facilities of large networks. Gateways are responsible for tunneling (encapsulating) traffic. VPNs are widely used by enterprises to create wide area networks that span large geographic areas, to provide site-to-site connections to branch offices, and to allow mobile users to dial up their company LANs.

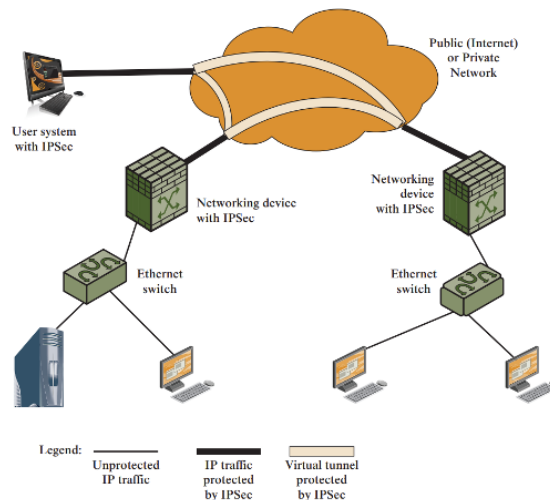


Figure 31: An IPSec VPN Scenario

Figure 31 is a typical scenario of IPsec usage. An organization maintains LANs at dispersed locations. Nonsecure IP traffic is conducted on each LAN. For traffic offsite, through some sort of private or public WAN, IPsec protocols are used. These protocols operate in networking devices, such as a router or firewall, that connect each LAN to the outside world. The IPsec networking device will typically encrypt all traffic going into the WAN and decrypt traffic coming from the WAN; these operations are transparent to workstations and servers on the LAN. Secure transmission is also possible with individual users who dial into the WAN. Such user workstations must implement the IPsec protocols to provide security.

3.4 Practice Quiz

Question 1: This question is about IP subnetting. We briefly covered this in the lecture. If you are still getting familiar with the concept, use this link to read more about it. Also, understand the ideas,

such as the network address, broadcast address, and the default gateway, if you still need to learn them.

<https://learn.microsoft.com/en-us/troubleshoot/windows-client/networking/tcpip-addressing-and-subnetting>

You have an IP address of 172.16.13.5 with a 255.255.255.128 subnet mask. What is your class of address, subnet address, and broadcast address?

- a) Class A, Subnet 172.16.13.0, Broadcast address 172.16.13.127
- b) Class B, Subnet 172.16.13.0, Broadcast address 172.16.13.127**
- c) Class B, Subnet 172.16.13.0, Broadcast address 172.16.13.255
- d) Class B, Subnet 172.16.0.0, Broadcast address 172.16.255.255

Explanation: We know that the prefix 172 lies in class B (128 to 191) of IPv4 addresses. From the subnet mask, we get that the class is divided into 2 subnets: 172.16.13.0 to 172.16.13.127 and 172.16.13.128 to 172.16.13.255. The IP 172.16.13.5 lies in the first subnet. So the starting address 172.16.13.0 is the subnet address, and last address 172.16.13.127 is the broadcast address.

Question 2: Again, this is a question to refresh your networking knowledge. If you are still getting familiar with the concept, use this link to read more about it. Also, understand the ideas, such as the network address, broadcast address, and the default gateway, if you still need to learn them.

<https://learn.microsoft.com/en-us/troubleshoot/windows-client/networking/tcpip-addressing-and-subnetting>

If a host on a network has the address 172.16.45.14/30, what is the subnetwork this host belongs to?

- a) 172.16.45.0
- b) 172.16.45.4
- c) 172.16.45.8
- d) 172.16.45.12**

Explanation: By prefix, we know that this is a class B address. First, we convert it to binary.

10101100.00010000.00101101.00001110 Next, we know that 30 bits are used for the network - so the network address is

10101100.00010000.00101101.00001100

Converting back to decimal, we get

172.16.45.12

Therefore, the answer is D.

Question 3: is a collection of protocols designed by IETF (Internet Engineering Task Force) to provide security for a packet at the network layer.

- a) IPSec**
- b) SSL
- c) PGP
- d) None of these

Explanation: The Answer is IPSec, SSL protects the transport layer, and PGP protects the application layer. In one of the slides, we discussed that IPSec is defined through IETF RFCs, and there is an IETF taskforce for IPSec.

Question 4: is a one-way relationship between a sender and a receiver that affords security services to the traffic carried on it.

- a) SAD
- b) SPD
- c) SA
- d) SPI

Explanation: This is the definition of a security association.

Question 5: The SSL Internet standard version is called

- a) SSH
- b) HTTP
- c) SLP
- d) TLS

Explanation: While some still use the historical term SSL, the current internet versions are called as TLS (Transport Layer Security).

Question 6: The is used to convey TLS-related alerts to the peer entity.

- a) Change Cipher Spec Protocol
- b) Alert Protocol
- c) SSL Record Protocol
- d) Handshake Protocol

Explanation: It is the alert protocol. The question gives away the answer. However, it is good to refresh the memory of what each protocol does.

Change Cipher Spec Protocol - Lets the other party know that it has generated the session key and is going to switch to encrypted communication

Alert Protocol - The Alert Protocol conveys SSL-related alerts to the peer entity.

SSL Record Protocol - This is the base protocol on top of which other protocols operate.

Handshake Protocol - This protocol allows the server and client to authenticate each other and to negotiate an encryption and MAC algorithm and cryptographic keys to be used to protect data sent in an SSL record.

Question 7: The Change Cipher Spec Protocol is one of the four TLS-specific protocols that use the SSL Record Protocol. TRUE or FALSE.

- a) TRUE

b) FALSE

Explanation: The structure of TLS is like this below. So, the answer is correct.

TLS subprotocols



Figure 32: TLS protocol structure

Question 8: Both tunnel and transport modes can be accommodated by encapsulating security payload encryption format. TRUE or FALSE.

- a) TRUE
- b) FALSE

Explanation: Both tunnel and transport modes can use ESP.

Question 9: In IPSec, Authentication must be applied to the entire original IP packet. TRUE or FALSE.

- a) TRUE
- b) FALSE

Explanation: This is false because, for example, in AH, only parts of the IP datagram are authenticated.

Question 10: Transport mode provides protection to the entire IP packet.

- a) TRUE
- b) FALSE

Explanation: Transport Mode authenticates and encrypts only the payload. The IP header is only authenticated, not encrypted. So the entire IP datagram is not protected.